DEEP LEARNING
UNIT – III | COMPLETE ANSWERS

Advanced Optimization (AdaGrad, RMSProp, Adam, NAG), Batch Normalization, Dropout & DropConnect

B.Tech V Semester – AI & ML | MIC23 | Subject Code: 23AM5T01

PART A	PART B	PART C
11 short answers	9 expanded 10-mark answers	1 worked numerical

Prepared in the same exam-oriented format as Units I and II. All questions in Unit III of the supplied question bank are covered.

UNIT III – PART A: SHORT ANSWER QUESTIONS (2 MARKS EACH)

Q1. Main advantage of Adaptive Optimization over SGD

Adaptive optimizers automatically adjust the effective learning rate for each parameter using past gradients. They often provide faster and more stable convergence when parameters have different gradient scales. Examples: AdaGrad, RMSProp and Adam.

Q2. Role of Learning Rate

The learning rate (η) controls the size of each parameter update. If too high, training may overshoot the minimum or diverge. If too low, learning becomes very slow and may get stuck on flat regions.

Q3. Hessian Matrix

The Hessian is a square matrix of second-order partial derivatives: H■■=∂²L/(∂w■∂w■). It describes loss-surface curvature and is used by second-order methods such as Newton's method.

Q4. Batch Normalization and benefits

Batch Normalization normalizes layer activations using mini-batch mean and variance, then applies learnable scale and shift. It stabilizes training, permits higher learning rates and often accelerates convergence.

Q5. RMSProp

RMSProp maintains an exponentially weighted moving average of squared gradients. It divides the current gradient by the root of this average to prevent the effective learning rate from continually shrinking as in AdaGrad.

Q6. First and Second Moments in Adam

The first moment (m■) is the moving average of gradients and represents direction. The second moment (v■) is the moving average of squared gradients and represents their magnitude/scale.

Q7. Dropout and retention probability p

Dropout randomly disables neurons during training to reduce overfitting. Retention probability p is the probability a neuron remains active; dropout probability is 1−p.

Q8. DropConnect

DropConnect randomly sets individual weights to zero during training. Dropout removes neuron activations, whereas DropConnect removes connections/weights.

Q9. Saddle Point

A saddle point has approximately zero gradient, but the loss curves upward in some directions and downward in others. It is neither a local minimum nor a local maximum.

Q10. Momentum

Momentum combines the current gradient with a fraction of the previous update. It builds velocity in consistent directions and reduces oscillations in directions with rapidly changing gradients.

Q11. Optimization methods in chronological order

A common progression is Batch/Stochastic Gradient Descent → Momentum → Nesterov Accelerated Gradient → AdaGrad → AdaDelta → RMSProp → Adam.

PART B – ESSAY & LONG ANSWER QUESTIONS (10 MARKS EACH)

Q1. Explain Adam optimization with first/second moments, bias correction and parameter updates.

Introduction: Adam (Adaptive Moment Estimation) combines Momentum and RMSProp. It uses moving averages of gradients and squared gradients to calculate an adaptive update for every parameter.

1. First-moment estimate

The first moment is the exponentially weighted average of past gradients. It smooths the gradient direction, similar to momentum.

```
m■ = β■m■■■ + (1−β■)g■
```

2. Second-moment estimate

The second moment is the exponentially weighted average of squared gradients. It estimates gradient scale and adapts the step size.

```
v■ = β■v■■■ + (1−β■)g■²
```

3. Bias correction

Since m■ and v■ are initialized to zero, early estimates are biased towards zero. Adam corrects this bias:

```
m■■ = m■/(1−β■■) v■■ = v■/(1−β■■)
```

4. Parameter update

```
θ■ = θ■■■ − η × m■■/(√v■■ + ε)
```

θ is the parameter, η is learning rate, β■ and β■ are decay rates, ε avoids division by zero, and g■ is the current gradient. Common values are β■=0.9, β■=0.999 and ε=10■■.

5. Algorithm steps

• Initialize θ■, m■=0 and v■=0.

• Perform forward propagation and calculate loss.

• Compute gradient g■ using backpropagation.

• Update m■ and v■, then apply bias correction.

• Update parameters using the Adam equation; repeat until convergence.

Training batch	Forward pass + loss
Backpropagation → gradient	Update m■ and v■
Bias correction	Adaptive parameter update
Repeat	Trained network

6. Advantages and limitations

• Adaptive learning rates for individual parameters.

• Works well with noisy and sparse gradients.

• Combines momentum with RMSProp-like scaling.

• Requires extra memory for moment estimates; may need tuning and does not always generalize better than well-tuned SGD.

Conclusion: Adam is widely used because it provides efficient adaptive optimization through first- and second-moment estimates.

Q2. Explain RMSProp with equations, working and how it solves AdaGrad's decaying learning rate.

Introduction: RMSProp is an adaptive optimizer designed to overcome AdaGrad's limitation. AdaGrad accumulates all squared gradients, so its effective learning rate may become too small.

1. Exponential moving average

```
s■ = ρs■■■ + (1−ρ)g■²
```

g■ is the current gradient, s■ is the moving average and ρ is the decay factor (often around 0.9). Old gradients gradually lose influence.

2. Parameter update

```
θ■ = θ■■■ − η × g■/(√s■ + ε)
```

The denominator scales the update according to recent gradient magnitudes; ε provides numerical stability.

3. Working steps

• Initialize parameters and s■=0.

• Calculate loss and current gradient g■.

• Update moving average s■ using squared gradient.

• Scale the gradient by √s■+ε and multiply by η.

• Update parameters and repeat for each mini-batch.

Gradient g■
Square g■²
Moving average s■
Scale gradient g■/(√s■+ε)
Update θ

4. How it solves AdaGrad's limitation

AdaGrad uses G■=G■■■+g■². G■ continually increases, so η/√G■ continually decreases. RMSProp instead uses a decaying average; old gradients gradually lose influence, so the effective learning rate responds to recent gradients and does not shrink indefinitely.

5. Comparison

Feature	AdaGrad	RMSProp
History	All past squared gradients	Recent weighted history
Learning rate	Keeps decreasing	Adapts to recent scale
Memory	One accumulator/parameter	One moving average/parameter
Main limitation	May stop learning early	ρ and η need tuning

6. Advantages and limitations

• Adapts learning rates separately for parameters.

• Works well with noisy and non-stationary gradients.

• Avoids excessive learning-rate decay of AdaGrad.

• Requires choosing decay factor and initial learning rate.

Conclusion: RMSProp improves AdaGrad by replacing the unbounded sum of squared gradients with a decaying average.

Q3. Explain AdaGrad with equations, updates, advantages and vanishing learning-rate limitation.

Introduction: AdaGrad assigns a different effective learning rate to each parameter. Parameters with large or frequent gradients receive smaller updates; infrequent features can receive relatively larger updates.

1. Accumulation

```
G■ = G■■■ + g■²
```

G■ is the element-wise accumulated sum of squared gradients.

2. Parameter update

```
θ■ = θ■■■ − η × g■/(√G■ + ε)
```

η is the initial learning rate and ε is a small constant for numerical stability.

3. Algorithm

• Initialize θ■ and G■=0.

• Calculate loss for the current example or mini-batch.

• Compute gradient g■ by backpropagation.

• Accumulate squared gradient in G■.

• Scale the gradient by √G■+ε.

• Update parameters and repeat.

Loss function
Compute gradient g■
Accumulate G■=G■■■+g■²
Adaptive step ηg■/(√G■+ε)
Update weights θ■

4. Advantages

• Automatically adapts learning rate for every parameter.

• Useful for sparse data and infrequent features.

• Reduces need to manually set separate learning rates.

• Simple and computationally inexpensive.

5. Major limitation: vanishing effective learning rate

Since AdaGrad continually adds squared gradients, G■ never decreases. As training continues, the effective learning rate η/(√G■+ε) becomes progressively smaller. It may become so small that weights barely change, even when further learning is needed.

6. Applications and improvement

AdaGrad can be useful for sparse-feature problems such as text classification. RMSProp and AdaDelta address its decay problem using moving averages or bounded gradient history.

Conclusion: AdaGrad is useful for sparse gradients, but its continually decreasing learning rate limits long deep-network training.

Q4. Compare AdaGrad, AdaDelta and RMSProp in learning-rate adaptation, memory efficiency and practical performance.

Introduction: These adaptive optimizers modify effective learning rates using gradient history. Their main difference is how they retain and use that history.

1. AdaGrad

AdaGrad accumulates all past squared gradients: G■=G■■■+g■². Update: θ■=θ■■■–ηg■/(√G■+ε). It suits sparse gradients, but the learning rate may become too small.

2. RMSProp

RMSProp uses s■=ρs■■■+(1–ρ)g■² and updates θ■=θ■■■–ηg■/(√s■+ε). Old gradients lose influence, making it responsive to recent gradients.

3. AdaDelta

AdaDelta uses decaying averages of squared gradients and squared parameter updates. It scales the gradient using the ratio of their root-mean-square values, reducing dependence on a manually chosen global learning rate.

```
E[g²]■ = ρE[g²]■■■ + (1–ρ)g■²
RMS[Δθ]■■■ = √(E[Δθ²]■■■ + ε)
Δθ■ = – RMS[Δθ]■■■ / √(E[g²]■ + ε) × g■
E[Δθ²]■ = ρE[Δθ²]■■■ + (1–ρ)(Δθ■)²
```

4. Comparison table

Aspect	AdaGrad	RMSProp	AdaDelta
History	All past squares	Exponential average	Exponential averages
Learning rate	Continuously decreases	Recent gradient scale	Gradient + update history
Manual η	Required	Required	Less dependent
Memory	One accumulator	One average	Two averages
Strength	Sparse gradients	Non-stationary tasks	Less η sensitivity
Weakness	Vanishing step	Needs tuning	May need tuning

5. Practical performance and similarities

AdaGrad is useful when gradients are sparse and training is not very long. RMSProp is useful when gradient scales change during training. AdaDelta was designed to reduce AdaGrad's learning-rate decay and sensitivity to a global learning rate. All three use past gradient information and can be used with mini-batch training.

Conclusion: AdaGrad uses complete history, whereas RMSProp and AdaDelta use decaying averages, making the latter more suitable for long training.

Q5. Explain saddle points in high-dimensional loss landscapes, causes, comparison with local minima and how optimizers help.

Introduction: A saddle point is a point where the gradient is zero or very small, but the loss increases in some directions and decreases in others. It is neither a local minimum nor a local maximum.

1. Mathematical condition

```
∇L(θ) = 0
```

At a saddle point, the Hessian has both positive and negative eigenvalues, indicating mixed curvature.

Direction A	Direction B
Positive curvature: loss rises	Negative curvature: loss falls
Not a local minimum	Not a local maximum

2. Causes

• Deep networks contain a very large number of parameters and dimensions.

• Non-convex loss functions create flat and curved regions.

• Interactions between weights produce different curvature directions.

• Symmetry and redundant parameters can create flat regions.

3. Why problematic

Near a saddle point, gradients may be very small, so gradient descent takes tiny steps and training becomes slow. In high-dimensional spaces, saddle points and plateaus can be encountered frequently. A local minimum has no nearby downhill direction, but a saddle point has downhill directions that the optimizer may fail to exploit.

4. Momentum

```
v■ = μv■■■ − η g■ θ■ = θ■■■ + v■
```

Momentum accumulates updates across iterations. It can maintain movement through shallow regions and reduce oscillations.

5. Adaptive optimizers

AdaGrad, RMSProp and Adam scale gradients using accumulated or moving-average statistics. This can improve progress when parameters have different gradient magnitudes. Adam combines momentum-like first moments with adaptive second-moment scaling.

6. Practical strategies

• Use momentum or Nesterov momentum.

• Use RMSProp or Adam when gradient scales vary.

• Choose suitable learning rate and mini-batch size.

• Use good initialization and monitor loss and gradients.

Conclusion: Momentum and adaptive optimizers can help maintain useful updates around saddle regions, though they do not guarantee that every saddle point will be avoided.

Q6. Define Hessian. Explain Newton-Raphson and Quasi-Newton methods and why exact Hessian computation is difficult in deep learning.

Introduction: The Hessian is a square matrix of all second-order partial derivatives of a scalar loss function. It describes curvature around current parameter values.

1. Definition

```
H(θ)■■ = ∂²L(θ)/(∂θ■∂θ■)
```

For n parameters, H is n×n. Diagonal entries describe curvature for individual parameters; off-diagonal entries describe interactions.

2. Newton-Raphson method

Newton's method uses gradient and Hessian information to calculate a curvature-aware update.

```
θ■■■ = θ■ − H(θ■)■¹ ∇L(θ■)
```

The inverse Hessian scales and rotates the gradient according to local curvature. Near a well-behaved optimum, this method can converge rapidly.

Current parameters θ■
Calculate gradient ∇L and Hessian H
Compute H■¹∇L update
Update θ■■■
Repeat until convergence

3. Quasi-Newton methods

Quasi-Newton methods avoid calculating the exact Hessian at every iteration. They build an approximation to the inverse Hessian from changes in parameters and gradients. BFGS is a common method; L-BFGS stores a limited history to reduce memory.

4. Why Hessian is intractable in deep learning

- Millions of parameters produce an n×n matrix.
- Storing it requires memory proportional to O(n²).
- Computing and inverting it is expensive.
- The Hessian may be singular or ill-conditioned.
- Mini-batch gradients make curvature estimates noisy.

5. First-order vs second-order

First-order	Second-order
Uses gradient ∇L	Uses gradient and Hessian
Lower memory/computation	Higher memory/computation
SGD, Momentum, Adam	Newton, BFGS, L-BFGS

Conclusion: Hessian information is useful, but exact second-order optimization is usually too expensive for large deep networks. Practical training therefore commonly uses first-order methods or limited-memory approximations.

Q7. Explain Batch Normalization with equations, training/inference behavior and why it accelerates convergence.

Introduction: Batch Normalization (BN) normalizes activations using the mean and variance of a mini-batch. Learnable scale and shift parameters then preserve the network's representational flexibility.

1. Mini-batch mean

$$\mu_B = (1/m) \Sigma x_i$$

2. Mini-batch variance

$$\sigma^2_B = (1/m) \Sigma (x_i - \mu_B)^2$$

3. Normalization

$$\hat{x}_i = (x_i - \mu_B)/\sqrt{(\sigma^2_B + \varepsilon)}$$

m is batch size and ε is a small constant to prevent division by zero.

4. Scale and shift

$$y_i = \gamma \hat{x}_i + \beta$$

Gamma (γ) is a learnable scale parameter and beta (β) is a learnable shift parameter.

Layer activations
Mini-batch mean and variance
Normalize: $(x-\mu)/\sqrt{(\sigma^2+\varepsilon)}$
Scale and shift: $\gamma \hat{x}_i+\beta$
Next layer / activation

5. Training behavior

During training, BN calculates mean and variance from the current mini-batch. It normalizes activations and updates running estimates of mean and variance for inference.

6. Inference behavior

During inference, BN generally uses stored running mean and variance instead of calculating statistics from a new mini-batch. The learned γ and β are still applied. This supports stable prediction even for a single input.

7. Benefits

- Stabilizes the scale and distribution of activations.
- Often permits higher learning rates.
- Can make training faster and more stable.
- Mini-batch noise may provide a mild regularization effect.
- Can reduce sensitivity to initialization.

8. Limitations

Very small or unrepresentative batches can produce poor statistics. Running statistics may differ from inference data. BN needs care in some sequence models and variable-batch settings.

Conclusion: BN improves stability by normalizing activations and learning a suitable scale and shift, often accelerating convergence.

Q8. Explain Momentum and how Nesterov Accelerated Gradient improves it. Include vector diagrams.

Introduction: Momentum accelerates gradient descent by adding a fraction of the previous update to the current update. It reduces oscillations and speeds movement in directions where gradients consistently point the same way.

1. Standard gradient descent

```
θ■ = θ■■■ − ηg■
```

It uses only the current gradient and may oscillate in steep directions while moving slowly in shallow directions.

2. Momentum equations

```
v■ = μv■■■ − ηg■
θ■ = θ■■■ + v■
```

v■ is velocity, μ is momentum coefficient (often near 0.9), η is learning rate and g■ is current gradient.

3. Working

• Consistent gradients accumulate and increase velocity.

• Alternating gradients partly cancel, damping oscillations.

• The optimizer moves faster through shallow regions.

Without momentum	With momentum
■ ■ ■ ■	■ → → →
Zig-zag updates	Smoother accumulated direction

4. Nesterov Accelerated Gradient (NAG)

NAG improves classical momentum by calculating the gradient after looking ahead in the direction of current momentum. It predicts where parameters are heading and corrects the update.

```
g■ = ∇L(θ■■■ + μv■■■)
v■ = μv■■■ − ηg■
θ■ = θ■■■ + v■
```

5. Momentum vs NAG

Momentum	NAG
Gradient at current θ	Gradient at look-ahead θ+μv
Uses accumulated velocity	Looks ahead and corrects direction
May overshoot before correction	Can respond earlier to changing slope

6. Advantages and limitations

Momentum speeds convergence and reduces zig-zag motion. NAG can correct direction earlier because it evaluates the gradient at a look-ahead position. Both require tuning; an excessively high learning rate can still cause divergence.

Conclusion: Momentum builds velocity from past updates, while NAG adds a look-ahead correction that can improve optimization.

Q9. Differentiate Dropout and DropConnect. Explain training and test phase behavior with schematic diagrams.

Introduction: Dropout and DropConnect are regularization methods that reduce overfitting by randomly removing parts of a neural network during training. They differ in what is removed.

1. Dropout

Dropout randomly sets selected neuron activations to zero during training. If p is the retention probability, each neuron is kept with probability p and dropped with probability 1−p.

```
r■ ~ Bernoulli(p) h■ = r ■ h
```

With inverted dropout, retained activations are scaled by 1/p during training so their expected magnitude is consistent at inference.

2. DropConnect

DropConnect randomly sets individual connection weights to zero during training. It masks entries of the weight matrix rather than removing an entire neuron's output.

```
M■■ ~ Bernoulli(p) W■ = M ■ W

y = f(W■x + b)
```

3. Schematic diagrams

Dropout	DropConnect
Input → neurons → output	Input → weighted links → output
Whole neuron activation disabled	Individual links/weights disabled
Neuron-level masking	Connection-level masking

4. Training phase

Dropout: A random neuron mask is generated. Selected activations are set to zero; backpropagation updates active paths. Different subnetworks are sampled across iterations.

DropConnect: A random mask is applied to the weight matrix. Some connections are temporarily removed, and backpropagation updates the remaining active connections.

5. Testing / inference phase

Random dropping is disabled at inference. With inverted dropout, no extra scaling is required at test time. With DropConnect, the full weight matrix is used with suitable expectation-preserving scaling or an equivalent inference procedure, depending on implementation.

6. Differences

Feature	Dropout	DropConnect
Removed element	Neuron activation	Individual weights
Mask applied to	Layer output	Weight matrix
Granularity	Neuron level	Connection level
Training	Zero activations	Zero weights
Inference	Full network; inverted scaling	Full/expected weights with suitable scaling
Effect	Reduces neuron co-adaptation	Regularizes individual connections

7. Advantages and limitations

Both reduce overfitting and encourage robust representations. Dropout is simple and widely used. DropConnect provides finer-grained regularization but may add computation. Excessive dropping can cause underfitting.

Conclusion: Dropout removes neuron activations; DropConnect removes individual weights. Both train randomly thinned subnetworks and improve generalization when appropriately configured.

PART C – NUMERICAL PROBLEM (SOLVE WITH WORKING)

Problem 1. AdaGrad Single-Step Weight Update

Question: Calculate updated weight w■ after one AdaGrad optimization step.

Parameter	Given value
Initial weight w■	2
Learning rate η	0.1
Gradient g■	2
Initial accumulator G■	0
Smoothing constant ε	0

Step 1: Calculate accumulator

```
G■ = G■ + g■² = 0 + 2² = 4
```

Step 2: Apply AdaGrad update

```
w■ = w■ − η × g■/(√G■ + ε)
```

Step 3: Substitute values

```
w■ = 2 − (0.1 × 2)/(√4 + 0)
w■ = 2 − 0.2/2 = 2 − 0.1
```

Final Answer: w■ = 1.9

Quick formula revision

• AdaGrad: G■=G■■■+g■²; θ■=θ■■■−ηg■/(√G■+ε).

• RMSProp: s■=ρs■■■+(1−ρ)g■²; θ■=θ■■■−ηg■/(√s■+ε).

• Adam: m■=β■m■■■+(1−β■)g■; v■=β■v■■■+(1−β■)g■².

• Batch Normalization: x■=(x−μ)/√(σ²+ε); y=γx■+β.

• Momentum: v■=μv■■■−ηg■; θ■=θ■■■+v■.

Exam tip: For a 10-mark answer, write definition, equations, explain symbols, draw the relevant diagram, describe working steps, add advantages/limitations or comparison, and finish with a conclusion.